

Programmer avec les suggestions orthographiques

Trier, filtrer, visualiser et extraire — contrats et composition de fonctions

Traduit de Bootstrap World (Fall 2026) — CC BY 4.0

Table des matières

1	Introduction	3
1.1	Lancement	3
2	Trier et réduire les tables	4
2.1	Découvrir <code>sort</code>	4
2.2	Les contrats	4
2.3	Découvrir <code>first-n-rows</code>	4
3	Composer avec les Cercles d'Évaluation	6
3.1	Pourquoi composer ?	6
3.2	Du Cercle au code	6
3.3	Composer dans l'autre sens	7
3.4	Composer avec <code>count</code> et <code>bar-chart</code>	7
4	Interface utilisateur pour notre correcteur	8
4.1	Accéder aux lignes : <code>row-n</code>	8
4.2	Mesurer des chaînes : <code>string-length</code>	8
4.3	Dessiner du texte : <code>text</code>	8
4.4	Fonctions d'images	9
4.5	Construire une interface — Cercle d'Évaluation	9
5	Visualiser les données	10
5.1	Table de fréquences : <code>count</code>	10
5.2	Diagramme en barres : <code>bar-chart</code>	10
5.3	Diagramme circulaire : <code>pie-chart</code>	10
5.4	Les vrais correcteurs renvoient des mots, pas des tables !	11
6	Synthèse	12

Introduction

Objectif

En s'appuyant sur les types de données simples, les élèves découvrent les fonctions Pyret pour travailler avec des tables, créer des visualisations et des images. Ils sont initiés aux **contrats** comme « mode d'emploi des fonctions » et aux **Cercles d'Évaluation** pour visualiser la composition de fonctions.

Lancement

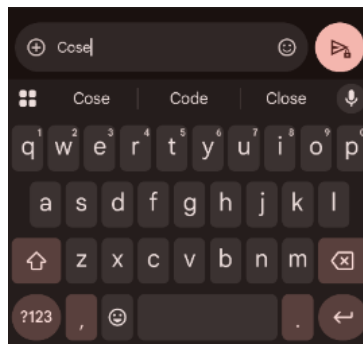


FIGURE 1 – Capture d'une appli de messagerie proposant des alternatives

Activité

Assurez-vous d'avoir ouvert le fichier `Correcteur-Orthographique-Starter.arr` et que votre Zone de Définitions contient :

```
resultats = alt-words("plation", WORDS-XS, 4)
```

Cliquez sur « Run ».

Notre correcteur nous donne des données sous forme de **tables** de mots suggérés avec leurs distances d'édition. On peut sauvegarder ces données avec des définitions de variables.

Réflexion

Peut-on faire quelque chose d'intéressant avec ces données ?

Un correcteur ne montre pas 500 suggestions, ni la distance d'édition. Mais il les *utilise* en interne pour sélectionner les mots les plus probables.

Activité

Essayez de taper "plation" dans un nouveau message texte ou document. Quelles alternatives votre appareil suggère-t-il ?

Trier et réduire les tables

Découvrir `sort`

Code

Dans la Zone d'Interactions :

```
– sort(resultats, "edit-distance", true)
– sort(resultats, "edit-distance", false)
– sort(resultats, "word", true)
```

La fonction `sort` prend trois **arguments** :

Argument	Type	Ce qu'il contient
1	Table	La table à trier
2	String	La colonne sur laquelle trier
3	Boolean	<code>true</code> = croissant, <code>false</code> = décroissant

Les contrats

Chaque **contrat** a trois parties :

1. **Nom** — le nom de la fonction
2. **Domaine** — le(s) type(s) des données qu'on donne
3. **Image** — le type de donnée produite

Point clé

Les contrats sont le **mode d'emploi** des fonctions. Le contrat de `sort` s'écrit :

```
sort :: Table, String, Boolean -> Table
```

On peut aussi annoter le Domaine avec des noms de variables :

```
sort :: (t :: Table, column :: String, ascending :: Boolean) -> Table
```

Découvrir `first-n-rows`

```
first-n-rows :: Table, Number -> Table
```

Code

```
– first-n-rows(resultats, 5)
– first-n-rows(sort(resultats, "edit-distance", true),
  10)
```

Activité

Complétez la section **Trier et réduire les tables** du cahier d'exercices.

Réflexion

Peu de suggestions ont une `edit-distance` de 1, mais beaucoup plus aux distances 2, 3 et 4. La croissance est presque exponentielle! Comment communiquer cela sans faire parcourir toute la table?

- Compter les suggestions par distance
- Faire un graphique

Composer avec les Cercles d'Évaluation

Objectif

Maintenant qu'on sait trier et réduire des tables, on peut **combinaison** ces fonctions pour obtenir exactement ce qu'on veut. Les **Cercles d'Évaluation** (*Circles of Evaluation*) sont l'outil visuel utilisé par Bootstrap pour représenter la composition de fonctions : chaque cercle contient une fonction et ses arguments, et les cercles s'imbriquent.

Pourquoi composer ?

Le tri seul donne une table de 500 suggestions classées. Ce qu'on veut, c'est **les 5 meilleures** — les plus proches du mot tapé. Pour ça, on a besoin de *deux* fonctions :

1. `sort(resultats, "edit-distance", true)` — trier par distance croissante
2. `first-n-rows(..., 5)` — garder les 5 premiers

La composition, c'est mettre la sortie d'une fonction dans l'entrée d'une autre.

Du Cercle au code

Voici comment on représente `first-n-rows(sort(resultats, "edit-distance", true), 5)` avec un Cercle d'Évaluation :

Le cercle extérieur (premier dans le code) : `first-n-rows`

Son 1^{er} argument est lui-même un cercle : `sort`

Ses arguments : `resultats, "edit-distance", true`

Son 2^e argument : `5`

→ `first-n-rows(sort(resultats, "edit-distance", true), 5)`

Point clé

La fonction **la plus à l'extérieur** du cercle est la **première** dans le code. On lit de l'extérieur vers l'intérieur.

Pour écrire le code : `first-n-rows( → sort(resultats, "edit-distance", true) → , 5)`

Composer dans l'autre sens

Réflexion

Que se passe-t-il si on inverse l'ordre? `sort(first-n-rows(resultats, 5), "edit-distance", true)` – est-ce que ça donne le même résultat que `first-n-rows(sort(resultats, "edit-distance", true), 5)`?

- **Non.** Le premier trie seulement 5 lignes (inutile), le second extrait les 5 meilleures après avoir tout trié (utile!).
- **L'ordre de composition compte.**

Composer avec `count` et `bar-chart`

Même principe pour visualiser : on ne passe pas la table brute à `bar-chart`, on passe d'abord par `count` pour obtenir les fréquences, *puis* on visualise.

```
bar-chart(count(resultats, "edit-distance"),  
          "edit-distance", "count")
```

↑ extérieur

↑ intérieur

↑ colonnes

Activité

1. Écris le code pour obtenir un **diagramme circulaire** des 10 premiers résultats triés par distance décroissante.
2. Dessine le Cercle d'Évaluation correspondant (combien de cercles emboîtés?).

Interface utilisateur pour notre correcteur

Objectif

On découvre comment extraire des valeurs d'une table, mesurer des chaînes, et créer des images avec Pyret — pour construire une interface qui affiche les suggestions orthographiques.

Accéder aux lignes : **row-n**

```
row-n :: Table, Number -> Row
```

Code

```
– row-n(resultats, 0) ["word"]      – premier mot suggéré  
– row-n(resultats, 0) ["edit-distance"]  – sa distance  
– row-n(resultats, 1) ["word"]      – deuxième mot suggéré
```

Point clé

`row-n(t, n) ["colonne"]` s'appelle un **accesseur de ligne** (*row accessor*). Il extrait la valeur d'une cellule spécifique.

Mesurer des chaînes : **string-length**

```
string-length :: String -> Number
```

Code

```
string-length("bonjour")      → 7  
string-length(row-n(resultats, 0) ["word"])  – longueur du  
1er mot
```

Dessiner du texte : **text**

```
text :: String, Number, String -> Image
```

Code

```
text("Bonjour !", 50, "blue")    — texte bleu de taille 50
```

Fonctions d'images

Plusieurs fonctions produisent des Image :

Contrat	Description
<code>triangle :: Number, String, String -> Image</code>	Triangle (taille, couleur, style)
<code>circle :: Number, String, String -> Image</code>	Cercle (rayon, couleur, style)
<code>star :: Number, String, String -> Image</code>	Étoile (rayon, couleur, style)
<code>rectangle :: Number, Number, String, String -> Image</code>	Rectangle (largeur, hauteur, couleur, style)
<code>ellipse :: Number, Number, String, String -> Image</code>	Ellipse (largeur, hauteur, couleur, style)
<code>regular-polygon :: Number, Number, String, String -> Image</code>	Polygone régulier (nombre de côtés, rayon, couleur, style)
<code>overlay :: Image, Image -> Image</code>	Superpose deux images
<code>rotate :: Number, Image -> Image</code>	Fait pivoter une image

Construire une interface — Cercle d'Évaluation

Activité

Objectif : superposer le mot "Platoon" (noir, taille 20) au-dessus d'un rectangle gris plein de 150×50.

1. Dessine le Cercle d'Évaluation (fonction extérieure : `overlay`)
2. Convertis-le en code :

```
overlay(  
  text("Platoon", 20, "black"),  
  rectangle(150, 50, "solid", "gray"))
```
3. **Sois créatif·ve** ! Ajoute de la couleur, plusieurs formes, voire plusieurs suggestions !

Visualiser les données

Table de fréquences : **count**

```
count :: Table, String -> Table
```

Code

```
count(resultats, "edit-distance")
```

`count` produit une table de fréquences : combien de suggestions pour chaque distance.

Diagramme en barres : **bar-chart**

```
bar-chart :: Table, String -> Image
```

Code

```
bar-chart(count(resultats, "edit-distance"),  
"edit-distance", "count")
```

Réflexion

- Passe le curseur sur une barre. Quelle information obtiens-tu ?
- Essaie `bar-chart` avec la colonne `"word"`. Est-ce utile ? Pourquoi ?

Diagramme circulaire : **pie-chart**

```
pie-chart :: Table, String -> Image
```

Code

```
pie-chart(resultats, "edit-distance")
```

Les vrais correcteurs renvoient des mots, pas des tables !

Activité

1. Dans la Zone de Définitions, ajoute : `ligne2 = row-n(resultats, 1)`. Clique « Run ».
2. Évalue `ligne2`. Qu'obtiens-tu ?
3. Évalue `ligne2["word"]` et `ligne2["edit-distance"]`.
4. Comment obtenir la distance d'édition du 5^e mot suggéré ?

Synthèse

Réflexion

- `first-n-rows` et `last-n-rows` ont le même Domaine et la même Image, mais font des choses différentes. Comment est-ce possible?
 - Ce dont une fonction a **besoin** $\neq$ ce qu'elle **fait**.
- Pourquoi `sort` a-t-il besoin de plus d'arguments que les autres?
- Pourquoi un correcteur a-t-il besoin d'accesseurs de ligne?
 - On veut suggérer un *mot*, pas une ligne entière!

Résumé des fonctions

Fonction	Contrat
<code>sort</code>	<code>sort :: Table, String, Boolean -> Table</code>
<code>first-n-rows</code>	<code>first-n-rows :: Table, Number -> Table</code>
<code>last-n-rows</code>	<code>last-n-rows :: Table, Number -> Table</code>
<code>count</code>	<code>count :: Table, String -> Table</code>
<code>bar-chart</code>	<code>bar-chart :: Table, String -> Image</code>
<code>pie-chart</code>	<code>pie-chart :: Table, String -> Image</code>
<code>row-n</code>	<code>row-n :: Table, Number -> Row</code>
<code>string-length</code>	<code>string-length :: String -> Number</code>
<code>text</code>	<code>text :: String, Number, String -> Image</code>

Notes pour l'enseignant·e

Ces fonctions de table et d'image sont utilisées tout au long du programme Bootstrap. Les élèves les reverront dans les leçons sur les arbres de décision, la science des données, et bien d'autres contextes. Prenez le temps de vous assurer que chaque élève est à l'aise avec leur lecture, leurs contrats, et la composition via les Cercles d'Évaluation.

À propos

Ces supports ont été développés en partie grâce au soutien de la *National Science Foundation* (bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).



Bootstrap Community — CC BY 4.0.

Traduction française — juillet 2026.

Source : <https://www.bootstrapworld.org/materials/fall2026/en-us/lessons/programming-spelling-suggestions/>